

Cortical-Column Memory: A Thousand-Brains-Inspired Memory and Consensus Architecture for Resource-Constrained Multi-Agent Systems

System Design, Implementation, and Preregistered Evaluation Protocol

James Booth

Independent researcher

<https://github.com/jmsbooth/agent-cortical-column-memory>

September 3, 2026

v0.1.0 · Architecture and Preregistered Evaluation Protocol · Preprint, not peer reviewed

Abstract

Resource-constrained deployments cannot always purchase larger models, longer context, or unconstrained inference. This paper proposes Cortical-Column Memory (CCM), an engineering architecture that tests whether modular state surrounding a small model can recover selected capabilities associated with model scale. CCM translates four ideas associated with the Thousand Brains literature into explicit software contracts: navigable reference frames, local column models, provenance-bearing structured memory, and active evidence gathering coordinated by structured hypothesis consensus. The translation is an architectural hypothesis, not a claim of biological realism or cortical equivalence. This v0.1.0 release publishes the reference implementation, deterministic synthetic task environments, baselines, component ablations, resource accounting, statistical decision rules, and machine-readable run manifests before the registered empirical campaign is completed. It therefore exposes a falsifiable validation framework rather than reporting a completed empirical study. The registered questions distinguish architectural benefit from additional test-time computation and retain abstention, unresolved disagreement, and negative findings as valid outcomes.

Publication state. The registered experimental campaign has not been completed. This release publishes the architecture, implementation, hypotheses, baselines, metrics, and decision criteria before final data collection. The only executable measurements included at release time are software smoke-test artifacts, clearly labelled as such in the repository.

1 Introduction

Large models can be unavailable or undesirable in local, private, disconnected, edge, sovereign, and low-energy deployments. Increasing parameter count is therefore not the only systems lever. A complementary possibility is to place explicit state, information acquisition, and coordination mechanisms around a smaller model. That possibility is useful only if the resulting system is evaluated against conventional inference scaling with comparable resources.

This paper asks a narrow question:

Can explicit modular memory, structured reference frames, distributed hypothesis formation, active evidence gathering, and structured consensus recover capabilities that would otherwise require a larger model, and can they do so efficiently enough to matter under constrained compute, energy, latency, or deployment conditions?

A second question is mandatory: does CCM outperform conventional small-model inference techniques when total inference budget is controlled? A result of the form “multiple agents outperform one agent” is insufficient if the multiple-agent system simply consumes more calls, tokens, wall-clock time, energy, or money.

1.1 Contributions and boundaries

The conservative contributions of this release are:

1. an inspectable architecture specification for CCM;
2. an engineering translation of selected Thousand Brains-inspired ideas into reference frames, columns, structured memory, active evidence, and consensus contracts;
3. a dependency-free reference implementation with a deterministic synthetic adapter;
4. a preregistered evaluation matrix covering baselines, component ablations, task classes, resources, and statistical decisions; and
5. a reproducible validation harness that emits provenance-bearing traces and machine-readable results.

This release does not claim empirical superiority, biological equivalence, or that a symbolic smoke adapter represents a language-model capability measurement. The future empirical release must replace protocol-state language with measured results only after the registered campaign is complete.

1.2 Research questions

RQ1 asks whether CCM improves success relative to a single small-model baseline. RQ2 asks whether any improvement survives budget-controlled comparison with conventional multi-agent inference. RQ3 asks which mechanisms account for an observed effect. RQ4 asks whether task class moderates the effect, including a pure-reasoning control where external memory should provide little benefit. RQ5 asks whether CCM produces safer coordination outcomes by reducing false consensus and preserving calibrated abstention.

2 Background and terminology

2.1 Thousand Brains Theory and the project implementation

Hawkins, Ahmad, and Cui describe a theory in which cortical columns learn models of objects and their structure relative to reference frames [5]. Hawkins et al. later develop a broader framework involving grid-cell-like location representations and object-centric reference frames [6]. These are theoretical and hypothesis-generating contributions. They are not a specification for language-model agents.

The Thousand Brains Project (TBP) and its Monty implementation are a separate, evolving sensorimotor learning system. Monty is an open-source Python implementation whose current documentation and repository should be cited directly when discussing its capabilities [2, 7]. This paper does not reimplement Monty and does not imply that its mechanisms, TBT, HTM, and CCM are interchangeable.

2.2 HTM and sparse distributed representations

Hierarchical Temporal Memory (HTM) is an algorithmic lineage associated with sparse distributed representations (SDRs), sequence memory, and cortical-learning algorithms [1, 4]. SDRs have a precise representational meaning in that lineage: multiple active components jointly encode a value with sparse distributed activity. A conventional dense or sparse vector embedding is not automatically an SDR. The v0.1.0 CCM implementation uses structured symbolic relations and no embedding; it therefore does not claim an SDR implementation.

2.3 The engineering analogy

The mapping in this paper is intentionally one-way. The inspiration suggests modular local models, object- or state-relative representation, multiple partial hypotheses, and active interaction. CCM implements those ideas as data structures and interfaces. It does not assert that the interfaces reproduce neurons, layers, grid cells, or biological learning. Where the analogy stops, the engineering contract begins.

3 CCM architecture

3.1 System overview

Figure 1 is the canonical architecture. A task or environment supplies observations and available actions to an orchestrator. Each column has a local model, one reference-frame state, local observations, candidate hypotheses, and a specialization label. Columns exchange structured states rather than private chain-of-thought. An append-only memory records observations with provenance. Consensus either commits a compatible hypothesis, abstains on low confidence, or remains unresolved. If the evidence is insufficient and an action is available, the active policy chooses a discriminative observation and the loop continues.

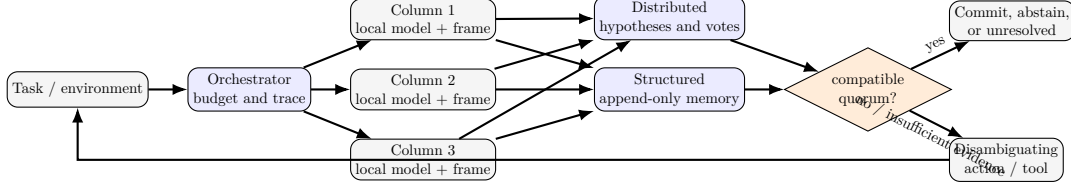


Figure 1: CCM architecture. The arrows represent explicit state and observation contracts; the diagram does not imply biological correspondence.

3.2 Reference frames as computation

A CCM reference frame is not a persona, topic, namespace, domain, or agent perspective. It is a navigable state space

$$F = (f, k, o, s, T, O),$$

where f is a stable identity, k is the frame kind, o is the origin, s is the current state, T is a set of validated transitions, and O maps locations to observation identifiers. A transition $(a, s_i, s_j) \in T$ is valid only when the current state is s_i ; an observation is attachable only when its frame and location are valid.

The first concrete example is spatial/object-like. A sensor observes a mug feature at object-local location `handle_left`; action `rotate_90` moves the state to `handle_top`; the frame predicts an observation at that location. The second is conceptual. A claim begins at `claim_unresolved`; action `inspect_source_a` moves to `claim_supported` when the source observation supports the claim; a competing claim remains open if the observation does not discriminate. Both examples contain a state, a transition, a predicted next location, and evidence associated with a location.

3.3 Columns

A column is a local model plus a bounded state owner. It is not merely one agent or one prompt. Its minimum state contains local observation identifiers, a reference-frame state, one or more candidate hypotheses, evidence identifiers, confidence, uncertainty, provenance, requested next action, and received peer states. Columns may share a base model or use heterogeneous models; v0.1.0 exposes specialization labels but does not claim a trained specialization.

The standard interface is:

```
observe(observation)
update_hypotheses(candidates)
propose_action(available_actions)
emit_state()
receive_peer_state(peer_state)
vote()
commit(status)
```

The interface separates local update from communication. A peer receives a serializable state, not hidden reasoning. A column can be committed, abstained, or unresolved by the orchestrator.

3.4 CCM memory model

CCM memory is neither conversation history, a scratchpad, ordinary RAG, a vector index, nor an assertion that episodic, semantic, and procedural memory have been biologically reconstructed. It is a structured, append-only observation ledger. The v0.1.0 object schema is:

Field	Contract
object_id	Stable unique identity; duplicate writes are rejected.
content	Human-readable observation payload.
frame_id, location	Reference-frame identity and state/location.
source, provenance	Origin and trace link; required for every object.
sequence, timestamp	Ordered run-local history; timestamps are stored at run level.
confidence, validation_state	Explicit quality and external validation status.
relations	Structured relations such as support or contradiction.
embedding	Optional future field; not used by the v0.1.0 symbolic adapter.

Historical observations cannot be silently mutated. A model-generated hypothesis is not accepted as an external observation. Conflicting observations remain separate records and are visible to consensus and the run trace.

3.5 Structured voting

A vote is a tuple containing `vote_id`, `column_id`, `hypothesis_id`, `hypothesis`, and confidence. A hypothesis contains a claim, frame, location, evidence identifiers, provenance, confidence, uncertainty, and optional requested action. Two hypotheses are compatible only when their normalized claim, frame identity, and location agree. Structured consensus sums confidence within compatible groups, requires a minimum quorum and confidence, and requires a margin over the runner-up. Ties and low-confidence groups are not forced into an answer. This differs from majority voting, self-consistency, debate, confidence-weighted answer aggregation, and judge-model aggregation because the primary unit is a frame-associated hypothesis with explicit evidence.

3.6 Active evidence

The active loop maintains competing hypotheses, identifies missing support, scores available actions by expected information gain divided by cost, executes one valid action, appends the resulting observation, and redistributes state. An action is not an answer and cannot create evidence without an environment response. If the action budget is exhausted, the system terminates with the best permitted non-commit outcome. The harness records action selection, returned observation, cost, and whether required evidence was retrieved.

Table 2: CCM termination outcomes.

Outcome	Condition
Committed	Compatible structured quorum, confidence, and margin are met.
Abstained	No safe commitment; aggregate confidence is below threshold.
Unresolved	Competing hypotheses remain too close or lack quorum.

3.7 Primary algorithm

Algorithm 2 is implementation-independent pseudocode. It makes termination and disagreement explicit.

1. Initialize N columns, their reference frames, an empty append-only memory, and a run manifest.
2. Observe the task/environment and write only externally returned observations with source and provenance.
3. Each column updates local hypotheses and emits a structured state.
4. Exchange states and group compatible hypotheses by frame, location, and claim.
5. If a group satisfies quorum, confidence, and margin criteria, commit it.
6. Otherwise, if uncertainty remains and an action budget is available, choose the highest information-gain-per-cost action.
7. Execute the action, validate its returned observation, append it, and repeat from step 3.
8. If evidence is insufficient, confidence is below threshold, or the budget is exhausted, abstain or terminate unresolved.

3.8 Invariants

The implementation and tests enforce the following invariants where feasible: every memory object has provenance; every hypothesis has a reference frame; every vote references an existing hypothesis; historical observations cannot be overwritten; disagreement is retained with its resolution mechanism; model-generated information cannot be represented as externally observed evidence; and each run has a deterministic configuration identity plus model, task, hardware, seed, and git metadata.

4 TPAA integration context

The original manuscript placed CCM “under TPAA.” In this release TPAA is retained as an integration context rather than placed in the title or treated as prerequisite knowledge. CCM can be

evaluated as a module inside a broader multi-agent architecture with an orchestrator, tool boundary, event transport, and deployment policy. The paper’s contribution remains understandable without TPAA: the testable unit is the CCM memory, reference-frame, column, active-evidence, and consensus protocol.

The integration boundary is intentionally narrow. An orchestrator supplies a task, model adapter, available actions, budgets, and trace sink. A column consumes observations and peer states and emits structured state. A tool returns an observation or an explicit failure. The transport may be local function calls or a future RPC/event layer; the v0.1.0 implementation uses local calls and has no cross-service deployment claim. This keeps the validation harness focused on architecture rather than vendor or topology lock-in.

5 Preregistered experimental design

The authoritative machine-readable registration is in `preregistration/`. The full registered configuration is `preregistration/registered-configs/ccm-v0.1.0.json`; the smoke configuration is separate and is not a substitute for the campaign.

5.1 Baselines and ablations

ID	Definition
B0	One small model, direct inference.
B1	One small model with equivalent additional context/memory.
B2	Self-consistency over independent samples.
B3	Conventional multi-agent system using the same small model without aggregation.
B4	Conventional multi-agent system with answer-level majority vote.
B5	Answer-level confidence-weighted aggregation.
CCM-Full	Explicit frames, structured memory, distributed hypotheses, active evidence, structured consensus, specialization.
CCM-NoRF	Remove explicit reference-frame constraints.
CCM-NoActive	Remove active evidence acquisition.
CCM-NoVote	Commit from one highest-confidence column without structured consensus.
CCM-NoStructuredMemory	Disable append-only structured retrieval.
CCM-NoSpecialization	Use a shared specialization.

The goal is explanatory ablation, not merely a leaderboard. Every strategy sees the same task instances and reports the resources it actually consumes.

5.2 Task classes

The deterministic suite includes persistent memory, multi-hop evidence integration, active information acquisition, ambiguous hypothesis resolution, tool use, long-horizon execution, conflicting evidence, incomplete information, and pure reasoning. The last class is a control: the protocol does not presume that memory or coordination can substitute for intrinsic reasoning capability. Each task has a hidden answer, candidate hypotheses, an explicit reference frame, optional observations/actions, and a stable versioned task identifier.

5.3 Compute and resource control

Every task record includes input tokens, output tokens, total model invocations, wall-clock latency, communication bytes, and fields for CPU utilization, GPU utilization, peak memory, accelerator time, energy, and hosted API cost. Unavailable measurements are recorded as `null` with a note, not imputed. The analysis reports both an unconstrained capability view and budget-controlled comparisons across approximately equal token, call, wall-clock, dollar, and measurable energy budgets. The principal secondary output is a capability/resource Pareto frontier. A gain that disappears under matching resource budgets is not credited as architecture-only improvement.

The full campaign is designed to compare multiple model classes when hardware and licensing permit: an approximately 3–4B local model, an approximately 7–9B local model, a larger open model, and a hosted frontier reference. Model revisions, quantization, prompts, runtime, and hardware must be recorded. v0.1.0 does not download or redistribute any model.

5.4 Statistics and decision rules

The unit of analysis is a task instance, paired by task identifier and seed where possible. The registered configuration uses three independent seeds and ten task instances per class. Report raw effects, 95% percentile bootstrap confidence intervals with 10,000 resamples, and effect sizes. Use paired success-rate differences for binary task success and paired standardized mean differences for continuous measures. Holm correction is applied across the five primary pairwise comparisons. A minimum practically meaningful effect is 0.05 absolute task-success rate. Bounded metrics are not assumed to be normally distributed.

Failures and timeouts remain in the task-success denominator. Invalid manifests or missing traces invalidate a run and are reported rather than silently dropped. No outlier removal or data-dependent stopping is permitted. Exploratory failure distributions, Pareto frontiers, and interactions are labelled exploratory.

5.5 Failure taxonomy and provenance

The harness classifies unsuccessful outcomes where possible as base-model knowledge failure, retrieval failure, reference-frame localization failure, incorrect state transformation, memory contamination, evidence-selection failure, hypothesis-generation failure, consensus failure, premature convergence, overconfidence, tool failure, orchestration failure, context exhaustion, or compute-budget exhaustion. A completed task record retains the prompt, structured column states, observations, memory operations, actions, hypotheses, votes, final result, metric inputs, failure class, and resource record. It does not require private chain-of-thought.

6 Implementation

The repository is a standard-library-only Python reference implementation. The module boundary mirrors the architecture: `ccm/reference_frames` validates state transitions; `ccm/memory` owns

append-only observations; `ccm/column` implements the column interface; `ccm/consensus` implements structured and baseline voting; `ccm/active_evidence` selects actions; `ccm/orchestration` runs the loop; `experiments` supplies deterministic worlds and strategies; and `evaluation` emits metrics, resource records, and reports.

The model boundary is explicit. `ToyModel` is a deterministic symbolic adapter for smoke validation and unit tests. It is useful because it makes hidden truth, evidence, and failure behavior inspectable, but it is not a claim about a 3B, 7B, or frontier language model. A future adapter can implement the proposal callback while preserving the state contracts and manifest requirements.

6.1 Configuration and run identity

Experiments are configuration-driven JSON files. Configuration includes experiment and protocol IDs, seed set, task classes, model family/version/revision, quantization, temperature, context window, column count and specializations, memory representation, consensus thresholds, action and step budgets, and exclusion rules. The runner hashes the canonical JSON configuration. Every raw JSONL result references that hash, the git commit, model version, task version, hardware metadata, seed, timestamp, and complete trace. Results without a manifest are invalid for registered analysis.

6.2 Machine-readable outputs

The runner writes one JSON object per task/strategy to `results/raw/` and a manifest to `results/manifests/`. The analysis command reads raw records, computes metrics without hand-copying values into LaTeX, writes JSON and CSV summaries, and generates the table source used by the paper when an analysis artifact is requested. Smoke output is committed only to demonstrate the path; registered output is not represented as completed evidence in this release.

6.3 Security and operational limits

The harness is offline by default and has no credential path. Model outputs are untrusted data. Provenance fields distinguish external observations from model hypotheses. The implementation avoids raw sensitive payloads and private reasoning. Resource fields that cannot be measured on the host remain unavailable. A production deployment would additionally require authenticated tool boundaries, trace propagation, bounded labels, and deployment-specific access controls; those are outside this local reference implementation.

7 Results status

This v0.1.0 paper does not report a completed registered empirical result. That is a declared publication state rather than a missing table. The registered campaign is pending, and no hypothesis is labelled supported or rejected here.

The repository does include a deterministic software smoke run. Its purpose is to verify that task generation, all requested protocol paths, manifests, provenance traces, resource records, metric generation, and generated table code execute on a clean checkout. Smoke values are not used as evidence for H1–H6 because the adapter is symbolic and intentionally exposes the task structure.

The v1.0 empirical release will replace this section only after all preregistered seeds and task instances are accounted for. It must report raw and processed results, confidence intervals, effect sizes, budget-controlled comparisons, ablations, failure taxonomy, resource Pareto analysis, deviations, and negative findings. Until then, claims in this paper are limited to design decisions, implementation facts, and the registered procedure.

8 Discussion

CCM makes a falsifiable systems proposition: some capability failures of a small model may be caused by missing state, missing evidence, or poor coordination rather than only by insufficient parameters. The proposition has two important constraints. First, the architecture must be evaluated against conventional inference scaling. Second, any observed gain may be local to task classes whose information structure matches the explicit memory and action contracts.

The design therefore treats success, calibrated abstention, unresolved disagreement, communication cost, and failure location as first-class outcomes. A system that avoids a confident unsupported answer by abstaining may be preferable to one that wins a forced-choice score, while a system that simply adds calls without improving a budget-controlled frontier is not a useful architectural result.

Several outcomes would be informative. Broad improvement over B0 but not B3 would suggest that coordination is useful while conventional multi-agent inference captures the effect. Improvement over B3 only on memory and active-observation tasks would support a narrower architectural claim. No improvement, or degradation from communication overhead and correlated errors, would be a valid result that constrains the proposal. The protocol is designed to retain each of these outcomes.

The architecture also makes a deliberate departure from the biological inspiration. Its reference frames are typed application state spaces, its columns are software objects around model calls, and its votes are explicit records. These choices improve inspectability and ablation, not biological fidelity.

9 Related work and threats to validity

9.1 Related work

The Thousand Brains theory supplies the principal architectural inspiration: local models, reference-frame-relative structure, sensorimotor interaction, and distributed agreement [5, 6]. TBP/Monty provides a contemporary open implementation of a sensorimotor learning system, but its mechanisms and task domain differ from CCM [2, 7]. HTM and SDR work provide a related representational lineage, not a language-agent implementation [1, 4].

Agent-memory work includes systems for long-lived context and generative agent state, such as

MemGPT and Generative Agents [8, 9]. These systems overlap with CCM in retaining state but do not by themselves define navigable frame transitions, structured peer hypotheses, or the registered active-evidence ablations. Self-consistency samples multiple reasoning paths and aggregates answers [11]; multi-agent debate exchanges arguments over rounds [3]; and mixture-of-agents combines layered agent outputs [12]. These are direct baselines because their gains can arise from additional test-time computation. Test-time scaling work further motivates reporting capability/resource frontiers rather than accuracy alone [10].

9.2 Novelty and distinction matrix

Approach	Local models	Explicit frame	Structured persistent memory	Active evidence / structured consensus
TBT theory	conceptual	object-centric	theoretical	sensorimotor / voting-inspired
TBP/Monty	modular sensorimotor	learned spatial/object state	learned model state	sensorimotor control
HTM/SDR	algorithmic modules	location-related lineage	sequence memory	not a language-agent consensus protocol
Memory agents	usually one or more agents	usually implicit	context/retrieval-centric	varies
Self-consistency / debate / MoA	repeated model calls	implicit	transient outputs	answer/argument aggregation
CCM	explicit columns	typed navigable state	provenance-bearing observation ledger	information-gain action and frame-compatible quorum

The distinguishing claim is not that CCM invented memory, multiple agents, voting, or test-time compute. The testable design claim is the combination of explicit navigable state, evidence-linked distributed hypotheses, active discriminative observation, and resource-controlled ablation in a small-model deployment protocol.

9.3 Threats to validity

- **Benchmark contamination and task selection.** Synthetic tasks have known ground truth but may be easier than ecological tasks. Future external tasks must document contamination and provenance.
- **Prompt and model sensitivity.** A result may depend on a prompt, base-model family, quantization, or adapter. Exact revisions and prompts must be versioned; multiple model classes are required before broad claims.
- **Correlated agents.** Reusing one base model can correlate errors, so column count is not equivalent to independent evidence. The protocol records model identity and includes a no-specialization ablation.
- **Test-time compute.** More columns, rounds, tokens, and tool calls can produce apparent gains. Budget-matched analyses and resource Pareto frontiers are mandatory.
- **Evaluator bias.** Judge models can favor fluent consensus. Primary metrics use deterministic ground truth where possible; judge-based evaluation is secondary and disclosed.

- **Hardware and energy error.** Wall-clock and utilization vary by host; unavailable energy is not estimated as measured energy. Hardware metadata and measurement status are retained.
- **Tool quality and orchestration.** Active evidence can look beneficial if the tool leaks the answer or harmful if it is unreliable. Tool observations, action costs, and tool failures are logged.
- **Biological correspondence.** TBT is an inspiration, not biological validation. No neuroscience claim follows from a CCM result.
- **Architecture overfitting.** A task suite can reward the mechanisms it was designed to test. The pure-reasoning and conflicting-evidence controls, preregistration, and future held-out tasks limit but do not eliminate this risk.

10 Conclusion

CCM is presented here as a concrete, inspectable distributed-agent architecture rather than a reconstruction of cortical computation. It translates selected Thousand Brains-inspired ideas into explicit reference frames, columns, structured memory, active evidence, and consensus contracts. The repository exposes those mechanisms, registers hypotheses and ablations, controls test-time resources, records provenance, and preserves negative outcomes as valid research results.

The v0.1.0 contribution is methodological transparency: a reader can inspect and run the architecture before the full empirical campaign is collected. Whether CCM recovers useful capability under constrained budgets, and where it fails to compensate for model scale, remains an empirical question for the registered evaluation and future release.

A Preregistration summary

This appendix summarizes the frozen protocol. The YAML and JSON files in `preregistration/` are authoritative for execution.

A.1 Hypotheses

ID	Registered statement
H1	CCM-Full improves task success over B0 on the full task suite.
H2	CCM-Full improves task success over B3 under comparable call and token budgets.
H3	Removing explicit reference frames reduces performance on persistent state transformation tasks.
H4	Removing active evidence reduces performance on disambiguation tasks.
H5	Structured consensus reduces false consensus relative to answer-level majority voting.

H6	CCM benefit varies by task class, with smaller gains on pure reasoning controls than on memory/evidence tasks.
----	--

A.2 Analysis commitments

Three independent seeds and ten instances per task class are registered. Task success is binary and bounded; abstention and unresolved outcomes are failures for this metric. Confidence calibration, abstention quality, false consensus, memory precision/recall, evidence utilization, convergence steps, communication overhead, failure categories, and resources are secondary outcomes. Report raw effects, 95% bootstrap confidence intervals, effect sizes, and practical significance; apply Holm correction across primary pairwise tests; retain timeouts and failures; perform no outlier removal or outcome-dependent stopping.

A.3 Negative-results and version policy

The registered results will be published whether the hypotheses are supported, qualified, or rejected. A component that contributes nothing will be reported. Corrections that do not alter registered elements may be v0.2.x releases with changelog entries. Changes to hypotheses, metrics, baselines, exclusions, or thresholds require a deviation record and a new protocol version. v1.0.0 requires all registered seeds, raw and processed data, generated tables and figures, matched-budget analyses, ablations, failure taxonomy, resource Pareto analysis, and evidence-based abstract/conclusion updates.

References

- [1] Subutai Ahmad and Jeff Hawkins. How do neurons operate on sparse distributed representations? A mathematical theory of sparsity, neurons, and semantic memory. *arXiv preprint arXiv:1601.00720*, 2016. <https://arxiv.org/abs/1601.00720>.
- [2] Viviane Clay, Niels Leadholm, and Jeff Hawkins. The Thousand Brains Project: A new paradigm for sensorimotor intelligence. *arXiv preprint arXiv:2412.18354*, 2024. <https://arxiv.org/abs/2412.18354>.
- [3] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023. <https://arxiv.org/abs/2305.14325>.
- [4] Jeff Hawkins and Subutai Ahmad. Hierarchical temporal memory including HTM cortical learning algorithms. Numenta technical report, 2010. <https://www.numenta.com/assets/pdf/whitepapers/hierarchical-temporal-memory-cortical-learning-algorithm-0.2.1-en.pdf>.
- [5] Jeff Hawkins, Subutai Ahmad, and Yuwei Cui. A theory of how columns in the neocortex enable learning the structure of the world. *Frontiers in Neural Circuits*, 11:81, 2017. <https://doi.org/10.3389/fncir.2017.00081>.

- [6] Jeff Hawkins, Marcus Lewis, Mirko Klukas, Scott Purdy, and Subutai Ahmad. A framework for intelligence and cortical function based on grid cells in the neocortex. *Frontiers in Neural Circuits*, 12:121, 2019. <https://doi.org/10.3389/fncir.2018.00121>.
- [7] Niels Leadholm, Viviane Clay, Scott Knudstrup, Hojae Lee, and Jeff Hawkins. Thousand-brains systems: Sensorimotor intelligence for rapid, robust learning and inference. *Neural Computation*, 38(6):845–896, 2026. <https://doi.org/10.1162/NECO.a.1508>.
- [8] Charles Packer, Vivian Fang, Shishir Bothmann, Lin Xu, P. S. Ammanamanchi, Juan Gonzalez, Ruslan Salakhutdinov, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. <https://arxiv.org/abs/2310.08560>.
- [9] Joon Sung Park, Joseph O’Brien, Carrie J. Cai, Meredith Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. <https://doi.org/10.1145/3586183.3606763>.
- [10] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. <https://arxiv.org/abs/2408.03314>.
- [11] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2203.11171>.
- [12] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024. <https://arxiv.org/abs/2406.04692>.